

Fase 1 — Análise e Mapeamento dos Testes

1. Documentos Analisados

Os documentos são testes de exame de Segurança Informática, abrangendo as épocas normais e de recurso dos anos letivos 2019/20, 2021/22 e 2023/24. Cada teste avalia uma variedade de tópicos, desde criptografia básica até protocolos avançados e modelos de controlo de acessos.

2. Mapeamento dos Temas e Conceitos por Teste

Teste 1: Época Normal, 2021/22

Temas Principais:

- **Cifras simétricas em bloco (DES, AES)**
 - Modos de operação (ECB, CBC, GCM)
 - Determinismo e IV (Initialization Vector)
 - Garantias de autenticidade e integridade (GCM)
 - **Certificados X.509 e TLS**
 - Estrutura e verificação de certificados
 - Handshake TLS (client_random, server_random)
 - Record Protocol e derivação de chaves
 - **JCA (Java Cryptography Architecture)**
 - Métodos `update` e `doFinal` na classe `Cipher`
 - **OAuth 2.0 e OpenID Connect**
 - Conceitos: cliente, servidor de autorização, access_token
 - Fluxo Authorization Code Grant
 - Parâmetros: scope, client_id, client_secret, state, callback
 - **Funções de Hash (MD5, SHA1)**
 - Vulnerabilidades e implicações em assinaturas digitais
 - **RBAC (Role-Based Access Control)**
 - Conceito de sessão e relação UA
 - **MAC vs Assinatura Digital**
 - Autenticação de cookies em aplicações web
-

Teste 2: Época Normal, 2019/20

Temas Principais:

- **Cifras simétricas em bloco e padding**
 - Limitações do padding com zeros
- **Cifra e integridade (AE — Authenticated Encryption)**
 - Esquema AE com cifra simétrica + hash
 - Vulnerabilidades de integridade
- **Certificados X.509**
 - Assinatura de certificados folha e cadeia de confiança

- Campos protegidos por cifra
 - **JCA**
 - Independência de algoritmos (exemplo de concretização)
 - **TLS (Handshake)**
 - Ataques de downgrade e detecção
 - Pre-master secret e recomendações RFC 7525
 - **Armazenamento de passwords**
 - Ataques de dicionário e pré-computação
 - Funções de hash e salt
 - **RBAC**
 - Política no Moodle e especificação de permissões
 - **OAuth 2.0 e OpenID Connect**
 - Fluxo Authorization Code Grant
 - Parâmetros: scope, access_token, id_token
-

Teste 3: Época Normal, 2023/24

Temas Principais:

- **Cifras simétricas e IV**
 - Necessidade de IV em modos de operação
 - Dimensão da chave vs dimensão do ficheiro
 - **Certificados X.509, TLS e JCA**
 - Certificados auto-assinados
 - Classe **Mac** e dimensão da marca
 - Record Protocol e integridade
 - **RBAC**
 - Conceitos de modelos de controlo de acessos
 - **RFC 7516 (JWE)**
 - Duas chaves: CEk (chave de cifra) e Ke (chave pública do destinatário)
 - Processo de decifra
 - **TLS**
 - Ataque de downgrade e detecção
 - Perfect Forward Secrecy
 - **Bitcoin**
 - Estrutura de blocos e hashes
 - Imutabilidade da cadeia
 - **Armazenamento de passwords**
 - Cifra simétrica e riscos
 - Mitigação com salt e funções de hash resistentes
 - **OAuth 2.0 e OpenID Connect**
 - Parâmetro scope
 - Endpoints: authorization, token, userinfo
-

Teste 4: Época de Recurso, 2023/24

Temas Principais:

- **Esquemas criptográficos**
 - Chaves simétricas vs assimétricas
 - MAC vs assinatura digital
 - Necessidade de padding em modos de bloco
 - **Certificados X.509 e JCA**
 - Confiança em certificados
 - Classe `TrustStore`
 - Relação entre certificados (emissor/receptor)
 - **OAuth 2.0 e OpenID Connect**
 - Semelhanças entre os protocolos
 - Parâmetro `scope` e `access_token`
 - Fluxo Authorization Code Grant
 - **RFC 7516 (JWE)**
 - Cifra com GCM e garantia de autenticidade
 - **TLS e HTTPS**
 - Sub-protocolos e confidencialidade de cookies
 - **Assinatura digital de PDFs**
 - Primitivas criptográficas
 - Distribuição segura de chaves (PKI)
 - **RBAC**
 - Política e permissões por sessão
-

Teste 5: Época de Recurso, Verão 2021/22

Temas Principais:

- **Cifras simétricas em bloco**
 - Modos de operação (ECB, CBC)
 - Reutilização de chaves
 - **Certificados X.509 e TLS**
 - Verificação de autenticidade
 - Handshake e random numbers
 - Ataques a CAs e MITM
 - **JCA**
 - Modos da classe `Cipher` (`ENCRYPT_MODE`, `DECRYPT_MODE`, `WRAP_MODE`, `UNWRAP_MODE`)
 - **Autenticação baseada em passwords**
 - Salt e dificuldade de ataques
 - **MAC vs Assinatura Digital**
 - Autenticação de cookies
 - **RBAC**
 - Relação PA e hierarquia de permissões
 - **OAuth 2.0 e OpenID Connect**
 - Fluxo Authorization Code Grant
 - Parâmetro `state` e proteção contra CSRF
-

3. Cruzamento dos Temas e Determinação dos Tópicos Centrais

Tópicos Mais Frequentes (por ordem decrescente de aparição):

1. Criptografia Simétrica e Assimétrica

- Cifras em bloco (DES, AES)
- Modos de operação (ECB, CBC, GCM)
- Chaves simétricas vs assimétricas
- Funções de hash (MD5, SHA1)
- MAC vs Assinatura Digital
- JCA e classe **Cipher**

2. Certificados e PKI

- Estrutura X.509
- Cadeias de confiança
- Handshake TLS
- Record Protocol
- RFC 7516 (JWE)
- RFC 7525 (recomendações TLS)

3. Protocolos de Autenticação e Autorização

- OAuth 2.0
- OpenID Connect
- Fluxo Authorization Code Grant
- Parâmetros: scope, client_id, client_secret, state, callback

4. Controlo de Acessos

- RBAC (Role-Based Access Control)
- Sessões e relação UA/PA/RH
- Modelos de permissões

5. Segurança de Aplicações Web

- Autenticação de cookies (MAC vs assinatura digital)
- Ataques: CSRF, XSS, MITM
- Armazenamento seguro de passwords

6. Vulnerabilidades e Ataques

- Colisões em funções de hash
- Ataques de dicionário e pré-computação
- Ataques de downgrade
- Perfect Forward Secrecy

7. Blockchain e Bitcoin

- Estrutura de blocos

- Imutabilidade da cadeia

Progressão Lógica Provável da Matéria

1. **Fundamentos de Criptografia** (Cifras simétricas/assimétricas, hash, MAC, assinatura digital)
2. **Certificados e PKI** (X.509, TLS, JCA)
3. **Protocolos de Segurança** (TLS, OAuth 2.0, OpenID Connect)
4. **Controlo de Acessos** (RBAC, sessões, modelos)
5. **Segurança em Aplicações Web** (Cookies, CSRF, XSS)
6. **Vulnerabilidades e Ataques** (Colisões, ataques de dicionário, MITM)
7. **Tópicos Avançados** (Blockchain, Perfect Forward Secrecy)

Temas com Menor Frequência ou Dúvidas

- **Bitcoin/Blockchain:** Aparece apenas no Teste 3, sem relação clara com os restantes tópicos.
- **RFC 7516 (JWE):** Mencionado nos Testes 3 e 4, mas sem aprofundamento nos restantes.
- **RBAC em detalhe:** Aparece em todos os testes, mas com exemplos práticos diferentes (Moodle, políticas formais).

Ligações entre Temas

- **Criptografia** é a base para **Certificados**, **TLS**, e **PKI**.
- **TLS** usa **certificados** e **criptografia simétrica/assimétrica** para garantir confidencialidade e integridade.
- **OAuth 2.0/OpenID Connect** dependem de **certificados** (PKI) e **assinaturas digitais** para autenticação.
- **RBAC** é aplicado em **segurança de aplicações web** e **gestão de sessões**.

Termos-Chave Recorrentes

- **IV (Initialization Vector)**
- **Salt**
- **Padding**
- **MAC (Message Authentication Code)**
- **Perfect Forward Secrecy**
- **Client/Server Random**
- **Access Token / ID Token**
- **Scope / State**
- **Hierarquia de Certificados**
- **RBAC / UA / PA / RH**

[ORDEM INCERTA]

- **Bitcoin/Blockchain:** Não há informação suficiente para determinar a sua posição na progressão da matéria.

- **RFC 7516 (JWE)**: Parece ser um tópico avançado, possivelmente no final do curso.

[VERIFICAR]

- **Dimensão das chaves**: Em alguns testes, menciona-se que as chaves simétricas são menores que as assimétricas, mas não há consenso sobre a dimensão exata ou se isso é uma regra geral.
 - **Uso de salt em passwords**: Em alguns testes, o salt é usado com funções de hash, mas não há exemplos práticos de como implementar isso em código.
-

Fase 2 — Produção do Material Didático

Estrutura Proposta para o Material

O material será organizado em **capítulos**, seguindo a progressão lógica inferida. Cada capítulo incluirá:

- **Definições e conceitos fundamentais**
 - **Explicações detalhadas**
 - **Fórmulas, regras e propriedades**
 - **Ligações entre tópicos**
 - **Exemplos práticos** (sem resolver exercícios dos testes)
-

Material Didático: Segurança Informática

1. Fundamentos de Criptografia

1.1. Cifras Simétricas e Assimétricas

Cifras Simétricas

- **Definição**: Usam a mesma chave para cifrar e decifrar.
- **Exemplos**: DES, AES, 3DES.
- **Vantagens**: Velocidade, eficiência.
- **Desvantagens**: Problema da distribuição segura da chave.

Aplicações:

- Cifra de ficheiros, comunicações (TLS).
- Modos de operação: ECB, CBC, GCM, CTR.

Modos de Operação

Modo	Descrição	IV Necessário?	Autenticidade
ECB	Cada bloco é cifrado independentemente	Não	Não

Modo	Descrição	IV Necessário?	Autenticidade
CBC	Cada bloco depende do anterior	Sim	Não
GCM	Combina cifra e MAC (autenticidade)	Sim	Sim
CTR	Transforma cifra em modo fluxo	Sim	Não

Regra prática: Nunca usar ECB para texto com padrões (ex: imagens).

Cifras Assimétricas

- **Definição:** Usam um par de chaves (pública/privada).
- **Exemplos:** RSA, ECC, ElGamal.
- **Vantagens:** Resolve o problema da distribuição de chaves.
- **Desvantagens:** Lentidão, chaves maiores.

Aplicações:

- Assinaturas digitais, troca de chaves (TLS), autenticação.

Relação entre chaves:

- **Assinatura Digital:** Privada cifra, pública decifra.
 - **Cifra:** Pública cifra, privada decifra.
-

1.2. Funções de Hash e MAC

Funções de Hash (MD5, SHA1, SHA256)

- **Propriedades:**
 - **Determinísticas:** Mesma entrada → mesma saída.
 - **Resistência a colisões:** Difícil encontrar ($x \neq x'$) tal que ($H(x) = H(x')$).
 - **Avalanche effect:** Pequenas mudanças na entrada alteram completamente a saída.
- **Aplicações:**
 - Integridade de ficheiros, assinaturas digitais, armazenamento de passwords.

Vulnerabilidades:

- **Colisões:** MD5 e SHA1 são inseguros (ex: COLLISION ATTACK).
 - **Pre-image attacks:** Encontrar (x) tal que ($H(x) = y$) (dado (y)).
-

MAC (Message Authentication Code)

- **Definição:** Função que produz uma marca de autenticidade usando uma chave secreta.
- **Exemplos:** HMAC, CMAC.
- **Propriedades:**
 - **Integridade:** Modificações são detetadas.
 - **Autenticidade:** Só quem tem a chave pode gerar a marca válida.

- **Diferença para Assinatura Digital:**
 - MAC usa uma **chave secreta partilhada**.
 - Assinatura digital usa uma **chave privada** (assimétrica).

Quando usar MAC vs Assinatura Digital:

MAC	Assinatura Digital
Chaves simétricas	Chaves assimétricas
Partilha de segredo	Não requer partilha de segredo
Não repúdio? Não	Não repúdio? Sim
Velocidade: Rápido	Velocidade: Lento

1.3. Assinaturas Digitais

- **Definição:** Usam criptografia assimétrica para garantir autenticidade, integridade e não repúdio.
- **Processo:**
 1. Calcula-se o hash da mensagem.
 2. Assina-se o hash com a chave privada.
 3. Envia-se a mensagem + assinatura.
 4. Receptor verifica com a chave pública.

Aplicações:

- Certificados X.509, emails assinados, transações blockchain.

Exemplo (RSA):

- **Assinar:** $(\text{assinatura}) = \text{Hash}(m)^d \mod n$
- **Verificar:** $(\text{Hash}(m) == \text{assinatura}^e \mod n)$

1.4. JCA (Java Cryptography Architecture)

- **API independente de algoritmo:** Permite usar diferentes implementações de criptografia.
- **Classes principais:**
 - **Cipher:** Cifra/decifra.
 - **Mac:** Gera MAC.
 - **MessageDigest:** Funções de hash.
 - **KeyPairGenerator:** Gera pares de chaves.
 - **SecretKey:** Chaves simétricas.

Exemplo de uso:

```
// Inicializar Cipher para cifra simétrica
Cipher cipher = Cipher.getInstance("AES/CBC/PKCS5Padding");
```



```
cipher.init(Cipher.ENCRYPT_MODE, secretKey, iv);  
byte[] ciphertext = cipher.doFinal(plaintext);
```

Métodos **update** e **doFinal**:

- **Porquê dois métodos?**
 1. **Eficiência**: Permite processar dados em *chunks* (útil para ficheiros grandes).
 2. **Segurança**: Evita que dados parciais sejam processados sem proteção.

2. Certificados e Infraestrutura de Chave Pública (PKI)

2.1. Certificados X.509

- **Estrutura**:
 - **Versão**: 1, 2 ou 3.
 - **Serial Number**: Identificador único.
 - **Algorithm**: Algoritmo de assinatura.
 - **Issuer**: Emissor (CA).
 - **Validity**: Período de validade.
 - **Subject**: Titular (ex: **CN=www.isel.pt**).
 - **Public Key**: Chave pública do titular.
 - **Extensions**: Campos opcionais (ex: Key Usage).

Exemplo de extensões:

- **Key Usage**: Assinatura, cifra, não repúdio.
- **Subject Alternative Name (SAN)**: Nomes alternativos (ex: **DNS:www.isel.pt**).

Hierarquia de Certificados

- **CA Raiz**: Auto-assinada, confiança universal.
- **CA Intermédia**: Assinada pela CA Raiz.
- **Certificado Folha**: Assinado pela CA Intermédia.

Processo de verificação:

1. Verificar assinatura do certificado folha com a chave pública da CA Intermédia.
2. Verificar assinatura da CA Intermédia com a chave pública da CA Raiz.
3. Verificar a cadeia de confiança até à CA Raiz.

Tipos de Certificados

Tipo	Descrição
Auto-assinado	Não é emitido por uma CA. Usado para testes ou servidores internos.
Folha	Emitido para utilizadores, servidores ou dispositivos.
Intermédio	Emitido por uma CA Raiz para delegar autoridade.

2.2. TLS (Transport Layer Security)

- **Objetivo:** Proteger comunicações na Internet (HTTPS).
- **Sub-protocolos:**
 1. **Handshake Protocol:** Negocia algoritmos, troca chaves, autentica servidor/cliente.
 2. **Record Protocol:** Transmite dados cifrados e autenticados.

Handshake Protocol

1. **Client Hello:** Cliente envia versões suportadas, suites de cifras, `client_random`.
2. **Server Hello:** Servidor escolhe versão/cifra, envia `server_random`, certificado.
3. **Key Exchange:** Troca de chaves (ex: Diffie-Hellman).
4. **Finished:** Ambos enviam um hash das mensagens trocadas para verificação.

Mecanismos de segurança:

- **Autenticação do servidor:** Via certificado.
- **Autenticação do cliente:** Opcional (certificado cliente).
- **Derivação de chaves:** Usa `client_random`, `server_random` e segredo partilhado.

Vulnerabilidades e defesas:

Ataque	Defesa
Downgrade (forçar algoritmos fracos)	Lista de suites permitidas, assinatura das mensagens de handshake
MITM (ataque ao certificado)	Validação da cadeia de confiança, pinning de certificados
Replay (repetir mensagens)	Nonce único (<code>client_random</code> , <code>server_random</code>)

Record Protocol

- **Responsabilidades:**
 - Confidencialidade: Cifra simétrica (AES-GCM).
 - Integridade: MAC ou assinatura digital.
 - Ordenação: Número de sequência.
- **Chaves usadas:** Derivadas durante o handshake.

2.3. RFC 7516 (JWE) e RFC 7525 (TLS)

RFC 7516 (JSON Web Encryption)

- **Objetivo:** Cifrar dados JSON de forma segura.
- **Processo:**
 1. Cifra o conteúdo com uma chave simétrica (**CEK**).
 2. Cifra a **CEK** com a chave pública do destinatário (**Ke**).
 3. Envia o criptograma e o IV.

Exemplo:

```
{
  "protected": "base64url(header)",
  "encrypted_key": "base64url(Ke(CEK))",
  "iv": "base64url(IV)",
  "ciphertext": "base64url(ciphertext)",
  "tag": "base64url(MAC)"
}
```

RFC 7525 (Recomendações para TLS)

- **Recomendações:**
 - Não usar RC4, 3DES, SHA-1.
 - Usar chaves assimétricas de ≥ 2048 bits (RSA) ou ≥ 256 bits (ECC).
 - Evitar o uso de chaves RSA para troca de chaves (usar Diffie-Hellman ou ECDHE).
 - Usar **Perfect Forward Secrecy (PFS)**.

3. Protocolos de Autenticação e Autorização

3.1. OAuth 2.0

- **Objetivo:** Delegar acesso a recursos sem partilhar credenciais.
- **Termos:**
 - **Resource Owner:** Utilizador que detém os dados.
 - **Client:** Aplicação que solicita acesso.
 - **Authorization Server:** Emite tokens.
 - **Resource Server:** Armazena os dados protegidos.
 - **Access Token:** Permissão temporária para aceder a recursos.

Fluxos (Flows)

Fluxo	Uso	Segurança
Authorization Code	Web apps, servidores	Alto (usando client_secret)
Implicit	Aplicações nativas (mobile)	Baixo (sem client_secret)
Client Credentials	Serviços máquina-máquina	Médio

Fluxo	Uso	Segurança
Resource Owner Password	Aplicações de confiança	Alto risco

Fluxo Authorization Code Grant (detalhado)

- Pedido de Autorização:**
 - Cliente redireciona utilizador para o servidor de autorização.
 - Parâmetros: `response_type=code`, `client_id`, `redirect_uri`, `scope`, `state`.
- Autorização do Utilizador:** Utilizador autoriza o acesso.
- Código de Autorização:** Servidor de autorização redireciona para `redirect_uri` com `code`.
- Troca do Código por Token:**
 - Cliente envia `code` + `client_secret` para o servidor de autorização.
 - Servidor responde com `access_token` e `refresh_token`.
- Acesso ao Resource Server:** Cliente usa `access_token` para aceder a recursos.

Parâmetros importantes:

- scope:** Define quais recursos podem ser acedidos.
- state:** Usado para prevenir CSRF (Cross-Site Request Forgery).
- client_id/client_secret:** Credenciais da aplicação cliente.

3.2. OpenID Connect

- Extensão do OAuth 2.0:** Adiciona autenticação de utilizadores.
- Termos:**
 - ID Token:** JWT que contém informações sobre o utilizador (ex: `sub`, `name`, `email`).
 - UserInfo Endpoint:** Endpoint para obter mais informações sobre o utilizador.

Diferenças para OAuth 2.0:

OAuth 2.0	OpenID Connect
Apenas autorização	Autenticação + autorização
Access Token	ID Token + Access Token
Sem informações do utilizador	ID Token contém <code>claims</code>

Proteção contra CSRF

- state parameter:** Valor aleatório gerado pelo cliente, incluído no pedido e verificado na resposta.
- Exemplo:**

```
GET /authorize?
response_type=code&client_id=CLIENT_ID&redirect_uri=REDIRECT_URI&scope=SCOPE&
state=STATE
```

- Servidor de autorização devolve:

```
GET /callback?code=CODE&state=STATE
```

- Cliente verifica se o `state` recebido é igual ao enviado.

4. Controlo de Acessos

4.1. RBAC (Role-Based Access Control)

- **Princípio:** Permissões são atribuídas a *roles*, não a utilizadores.
- **Componentes:**
 - **U (Utilizadores):** Quem acede ao sistema.
 - **R (Roles):** Conjuntos de permissões.
 - **P (Permissões):** Ações permitidas (ex: `read`, `write`).
 - **UA (User Assignment):** Associação utilizador-role.
 - **PA (Permission Assignment):** Associação role-permissão.
 - **RH (Role Hierarchy):** Hierarquia entre roles (ex: `admin > editor > viewer`).

Exemplo:

```
Roles: admin, editor, viewer
UA: { (Alice, admin), (Bob, editor) }
PA: { (admin, read/write/delete), (editor, read/write) }
```

Sessões em RBAC

- **Porquê sessões?**
 - Permite ativar um conjunto de roles para um utilizador num dado contexto.
 - Exemplo: Um utilizador pode ter o role `admin` num contexto e `viewer` noutro.
- **Implementação:**
 - Armazenar as roles ativas numa sessão.
 - Exemplo: `sessions = { (session_id, user_id, roles[]) }`

4.2. Modelos de Controlo de Acessos

Modelo	Descrição	Vantagens	Desvantagens
DAC (Discretionary)	Donos de recursos definem permissões	Flexível	Difícil gestão em grande escala
MAC (Mandatory)	Sistema define permissões (ex: militar)	Seguro	Pouco flexível

Modelo	Descrição	Vantagens	Desvantagens
RBAC	Permissões por roles	Equilibrado	Complexo em hierarquias profundas
ABAC (Attribute-Based)	Permissões baseadas em atributos	Muito flexível	Complexo de implementar

5. Segurança em Aplicações Web

5.1. Autenticação de Cookies

- **Cookies:** Usados para manter estado de sessão.
- **Ameaças:**
 - **Falsificação:** Envio de cookies manipulados.
 - **Replay:** Reutilização de cookies válidos.

Soluções: MAC vs Assinatura Digital

MAC	Assinatura Digital
Chave simétrica partilhada	Chave assimétrica (privada/pública)
Vantagens: Rápido, fácil de implementar	Vantagens: Não repúdio, não requer partilha de segredo
Desvantagens: Risco se a chave for comprometida	Desvantagens: Lento, chaves maiores
Uso: Cookies em aplicações internas	Uso: Cookies em aplicações de alto risco

Exemplo de assinatura de cookie:

```
import hmac
import hashlib

secret_key = b"minha_chave_secreta"
cookie_data = b"user_id=123;role=admin"
signature = hmac.new(secret_key, cookie_data, hashlib.sha256).hexdigest()

cookie_signed = f"{cookie_data};signature={signature}"
```

5.2. Ataques Comuns e Defesas

Ataque	Defesa	Exemplo
CSRF	state parameter, SameSite cookies	OWASP CSRF

Ataque	Defesa	Exemplo
XSS	CSP (Content Security Policy), escaping	OWASP XSS
MITM	TLS, certificados válidos	SSL Labs
Ataques de dicionário	Salt, funções de hash lentas (PBKDF2, bcrypt)	OWASP Password Storage

6. Armazenamento de Passwords

6.1. Problemas com Armazenamento Tradicional

- **Sem salt:** Permite ataques de dicionário e rainbow tables.
- **Cifra simétrica:** Se a chave for comprometida, todas as passwords são expostas.

Solução: Salt + Hash

- **Salt:** Valor aleatório único por utilizador.
- **Processo:**
 1. Gerar salt único para cada password.
 2. Armazenar: ($\text{hash} = H(\text{password} + \text{salt})$).
 3. Na autenticação: ($H(\text{password_input} + \text{salt}) = \text{hash armazenado}$).

Vantagens:

- Impede ataques de rainbow tables.
- Mesmo password → hashes diferentes.

Funções recomendadas:

- **PBKDF2, bcrypt, scrypt, Argon2** (resistentes a GPU/ASIC).

Exemplo de armazenamento seguro:

```
import hashlib
import os

def hash_password(password, salt=None):
    if salt is None:
        salt = os.urandom(16) # 16 bytes de salt aleatório
    hashed = hashlib.pbkdf2_hmac('sha256', password.encode(), salt, 100000)
    return salt + hashed # Armazenar salt + hash

# Exemplo de uso:
stored_password = hash_password("minha_password")
# Na autenticação:
salt = stored_password[:16]
hashed = stored_password[16:]
```

```
input_hashed = hashlib.pbkdf2_hmac('sha256', "minha_password".encode(), salt,
100000)
assert input_hashed == hashed
```

7. Vulnerabilidades e Ataques

7.1. Colisões em Funções de Hash

- **MD5 e SHA1:** Vulneráveis a colisões.
 - **Implicações:** Ataques como [SLOTH](#) em TLS.
- **Defesa:** Usar SHA256 ou superior.

7.2. Ataques de Downgrade

- **Objetivo:** Forçar o uso de algoritmos fracos (ex: RC4, 3DES).
- **Defesa:**
 - Lista de suites permitidas.
 - Assinatura das mensagens de handshake (RFC 7525).

7.3. Perfect Forward Secrecy (PFS)

- **Definição:** Garante que chaves de sessão não são comprometidas mesmo que a chave privada de longo prazo seja.
- **Como?:**
 - Usar chaves efêmeras (ex: ECDHE) para derivação de chaves de sessão.
- **Impacto:**
 - Apenas compromete sessões atuais se a chave privada for roubada.

8. Blockchain e Bitcoin (Tópico Avançado)

8.1. Estrutura de Blocos

- **Header do Bloco:**
 - **version:** Versão do protocolo.
 - **previous block hash:** Hash do bloco anterior.
 - **merkle root:** Hash das transações.
 - **timestamp:** Data/hora.
 - **nonce:** Valor usado na prova de trabalho.
 - **difficulty target:** Dificuldade da prova de trabalho.

Exemplo:


```
Bloco B_i:
previous_block_hash = H(B_{i-1})
transactions = [t1, t2, t3]
merkle_root = H(H(t1) || H(t2) || H(t3))
```

Imutabilidade da Cadeia

- **Prova de Trabalho (PoW):** Mineração de blocos.
- **Hash do bloco anterior:** Qualquer alteração invalida todos os blocos seguintes.

Ataque de alteração:

- Um atacante tenta alterar uma transação num bloco.
- **Consequência:** Todos os blocos seguintes terão hashes diferentes, invalidando a cadeia.
- **Conclusão:** Computacionalmente **infeasível** sem controle de > 50% da rede.

9. Resumo das Melhores Práticas

Área	Boas Práticas
Criptografia	Usar AES-GCM, chaves ≥ 128 bits (AES), SHA256+, evitar MD5/SHA1.
Certificados	Validar cadeia de confiança, usar certificados autoassinados apenas para testes.
TLS	Usar PFS (ECDHE), evitar versões antigas (SSLv3, TLS 1.0/1.1), usar suites fortes.
OAuth/OpenID Connect	Usar Authorization Code Grant, validar state , proteger client_secret .
RBAC	Definir roles claras, evitar hierarquias profundas, usar sessões para ativar roles.
Passwords	Usar salt único, funções resistentes (bcrypt, Argon2), nunca armazenar plaintext.
Cookies	Assinar ou usar MAC, definir HttpOnly , Secure , SameSite .
Ataques	Validar inputs (XSS), usar CSRF tokens, manter software atualizado.

Lista de Termos-Chave

- Cifras simétricas/assimétricas
- Modos de operação (ECB, CBC, GCM)
- IV (Initialization Vector)
- MAC (HMAC, CMAC)
- Assinatura digital (RSA, ECDSA)

- **Certificados X.509 (CA, folha, intermédio)**
 - **TLS (Handshake, Record Protocol, PFS)**
 - **OAuth 2.0 (Authorization Code Grant, access_token, scope)**
 - **OpenID Connect (ID Token, UserInfo)**
 - **RBAC (Roles, UA, PA, RH, sessões)**
 - **Salt e hash de passwords (bcrypt, Argon2)**
 - **CSRF, XSS, MITM, Downgrade**
 - **Blockchain (PoW, merkle root, imutabilidade)**
-